

ArchSync: Evidence-Backed Detection of Architecture Drift in TypeScript Systems

Vo Duc Hieu*
Software Engineering
FPT University
Ho Chi Minh City, Vietnam
voduchieu@littleboys.biz

Ha Hoang Bach
Information Assurance
FPT University
Ho Chi Minh City, Vietnam
bachcp6@littleboys.biz

Tran Minh Hoang
Computer Science and Engineering
VNUK Institute for Research and Executive Education,
The University of Danang
Da Nang, Vietnam
an1dee@littleboys.biz

Le Van Kiet
Software Engineering
VNUK Institute for Research and Executive Education,
The University of Danang
Da Nang, Vietnam
levankiet1212.2004@littleboys.biz

Abstract

Software teams can keep builds green while implementation relationships drift away from an approved architecture, leaving reviewers without a repeatable way to decide whether a change is harmless, forbidden, or a legitimate evolution. This paper presents ArchSync, a deterministic architecture-conformance prototype that treats a machine-readable model as an executable contract and reconstructs an observed service graph from TypeScript source code. Stack-specific analyzers detect selected HTTP, PostgreSQL, Redis, and AMQP interactions, preserve file-level evidence, and feed a Git-diff gate that maps introduced findings to PASS, BLOCK, or REVIEW without rewriting the approved model. We verify the prototype on two co-developed development datasets and replay the same controlled changes through the incremental gate. Within these finite regression sets, the implementation agrees with the declared graph, decision, evidence-location, and full-scan oracles, and provenance tracking removes the targeted detector errors exposed by the earlier analyzer. An executed dependency-cruiser inventory exposes an empty shared labeled subset, so it supplies no comparative accuracy estimate; the study also lacks an independently curated real-world holdout. Its results therefore establish executable-contract coherence and reproducible regression behavior only, not comparative advantage or general accuracy on TypeScript systems.

CCS Concepts

- **Software and its engineering** → **Software architectures**; *Software verification and validation*.

Keywords

software architecture, architecture conformance, architecture drift, static analysis, architecture as code, software evolution

1 Introduction

Software architecture defines intended components and permitted relationships. Evolution can introduce structural violations while compilation and functional tests remain green; mapping and practitioner studies describe erosion and continuing reliance on documentation and shared practices [3, 13]. A seemingly local change can bypass a gateway, access a database directly, or add infrastructure without an architecture decision.

ArchSync connects a versioned expected graph from `architecture.yaml` to an observed graph reconstructed from source. Explicit deny, allow, require, and require-path rules distinguish broken constraints from non-forbidden topology changes. Evidence links findings to implementation and model locations. For pull requests, only findings introduced relative to the Git merge base determine PASS, BLOCK, or REVIEW; historical findings remain visible. The observed graph follows code, while the expected graph remains approved intent. REVIEW requests a human decision and does not itself implement approval or rejection governance.

The prototype detects selected TypeScript/Node.js HTTP, PostgreSQL, Redis, and AMQP patterns. The model and conformance contracts are language-neutral, but only this source analyzer has empirical evidence. Determinism makes repeated explicit-rule decisions reproducible; it does not imply complete extraction or general accuracy. AI explanations, automatic repairs, IaC, and runtime observations remain outside the evaluated decision path.

The contributions are: (1) an executable typed architecture contract with four dependency-rule semantics; (2) a provenance-aware source-to-graph pipeline retaining file, line, column, detector, excerpt, and fixed confidence evidence; (3) a violation/evolution distinction with explicit decision precedence; (4) a cached, component-incremental Git-diff gate distinguishing introduced, historical, and resolved findings; and (5) a 20-patch development benchmark and 40-signal positive/hard-negative corpus with reproducible outputs. These are bounded prototype contributions grounded in prior conformance concepts, not claims of unprecedented rule semantics or validated industrial effectiveness.

*Corresponding author.

2 Background and Related Work

This scoped narrative synthesis establishes context for ArchSync’s design; it is not a systematic literature review. Sources were selected purposively from the retained keyword-search corpus for architecture conformance, drift detection, and static-analysis reliability, emphasizing 2022–2025. Four earlier works supply concept origins. Selection and interpretation may omit relevant approaches; coverage completeness and absence-of-prior-work claims are outside scope.

Erosion and reconstruction. A mapping of architecture erosion describes its manifestations and consequences [13]. Code-review studies identify symptoms in OpenStack and later OpenStack/Qt projects [12, 14]; their shared project contexts are not independent replications. Practitioner evidence identifies documentation, responsibilities, and shared practices as ways to manage drift [3]. Reconstruction taxonomies organize recovery strategies [7, 19], while a microservice mapping compares tool purposes and availability [4]. View-based analysis compares intended and reconstructed specifications in an industrial case [23]. ArchSync restricts its observation boundary to explicit service and infrastructure relationships.

Conformance. Reflexion Models, static conformance comparisons, and DCL provide the origins of model comparison and allowed/forbidden dependency rules [10, 17, 22]; neither concept is new here. CALint applies dependency-rule checks to Python [6]. ArchLintor is a close TypeScript comparison, detecting module-interaction violations and supporting CI/CD [20]. ArchSync’s typed service topology differs from that module boundary; no head-to-head accuracy result is claimed. IaC metrics address deployment coupling [18], and 3Erefactor searches for executable inconsistency-reducing repairs [15]. LLM-generated architectural tests distinguish execution success from correct rule expression [16]. ArchSync executes explicit rules deterministically, but that choice alone implies neither accuracy nor superiority.

Change-time feedback. Commit-level prediction studies architectural change [25]; repository mining tracks microservice evolution [8]; industrial conformance visualization supplies practitioner evidence [24]. A predicted or observed change need not violate intended architecture. ArchSync instead reports findings introduced at a Git head, preserving separate violation and evolution decisions.

Evaluation reliability. Replication support remains an evaluation concern [11]. Decomposition surveys cover an adjacent problem [1], whereas a microservice-recovery comparison executes nine tools and reports individual and combined recovery performance [21]. That study supplies an external-comparison precedent, not scores directly comparable with ArchSync. Static visual models and multiple dependency models motivate future multi-source work [2, 5]. Interrogation testing exposes both precision and soundness failures [9], motivating our explicit hard negatives. These sources establish comparison dimensions rather than exhaustive novelty or representative accuracy.

3 Problem Definition

Let $M = (C_E, E_E, R)$ contain expected components, typed directed edges, and rules. An edge is (u, t, v) : source, relation type, and target. For repository S , a stack-specific analyzer constructs $A(S) = (C_O, E_O, P)$, where P links observations to file, line, column, detector, excerpt, and a fixed confidence annotation (not a calibrated probability).

Deny rejects matching edges; *allow* restricts selected sources to an allowed target set; *require* demands a direct matching edge for every selected source; *require-path* demands a non-empty directed path. Selectors may use wildcards and constrain relation type. Given graph difference Δ and violation set $\mathcal{V}$, classification is

$$\text{class}(M, A(S)) = \begin{cases} \text{violation,} & |\mathcal{V}| > 0, \\ \text{evolution,} & |\mathcal{V}| = 0 \wedge |\Delta| > 0, \\ \text{no-impact,} & \text{otherwise.} \end{cases}$$

For Git base S_b and head S_h , findings are compared by stable semantic identity:

$$\mathcal{F}^+ = \mathcal{F}(M, A(S_h)) \setminus \mathcal{F}(M, A(S_b)).$$

Only introduced findings control the gate: a rule finding yields BLOCK, otherwise an evolution finding yields REVIEW, otherwise PASS. Common findings remain visible as pre-existing; base-only findings are resolved. Thus an unrelated patch need not repair historical drift before it can pass.

4 Research Questions

The feasibility study asks: **RQ 1**, how many expected components, relationships, and detector signals are reconstructed; **RQ 2**, whether no-impact/violation/evolution labels and violated rule sets match; **RQ 3**, whether findings contain the expected file and exact line; and **RQ 4**, whether normalized outputs replay identically and incremental/full-scan results agree, with what parsed scope and recorded cold/warm latency. D1 uses matched/extra/missed counts; D2 additionally uses descriptive signal-level precision, recall, F 1, and specificity. All questions are bounded by the declared development corpora.

5 Proposed Approach

Executable intent and evidence. A versioned YAML model defines component identifiers, layers, typed relationships, rules, severity, and ownership. Schema and semantic checks reject malformed models, unknown references, duplicate edges, and invalid identifiers. Expected edges are canonicalized; diagrams are derived views, not separate authorities. Findings retain a contract version, stable identifier, kind, severity, optional rule, affected element, source evidence, and model evidence. Forbidden-edge evidence identifies a detected call. Missing-edge/path evidence uses a deterministic component anchor because an absent call has no source location.

Observation. Guardian parses TypeScript under declared component roots with the Compiler API. Detectors inspect imports, variable initializers, calls, and endpoint expressions for selected HTTP, PostgreSQL, Redis, and AMQP patterns. PostgreSQL/Redis operations require a receiver associated with a recognized client;

AMQP requires a channel from a recognized connection. These per-file name associations filter unassociated method lookalikes but do not establish soundness under arbitrary reassignment or shadowing. Endpoints use literals and deterministic fallbacks, not runtime execution. Observations and evidence are sorted and deduplicated.

Fixed precedence. Any rule violation outranks evolution, regardless of severity; both finding types remain inspectable. This conservative policy prevents a forbidden edge from being treated only as a reviewable addition, but can delay acceptable changes bundled with it. Configurable severity/context policies are alternatives, not evaluated features. The command returns 0/1/3 for PASS/BLOCK/REVIEW. A repository must separately enforce those outcomes.

Incremental gate and review lifecycle. Guardian resolves a merge base, maps changed paths to source components, and caches the baseline graph by base commit, model hash, and analyzer version. It reparses all TypeScript files in affected components, replacing their observations while retaining unaffected ones. Component-wide scanning preserves independently detectable relationships; it does not add inter-file data-flow analysis. Core evaluates base and head with the same semantics before introduced findings determine the gate. Source annotations and a report support inspection.

The analyzer never rewrites the expected model. Accepting evolution requires a separately reviewed model change. The evaluated gate neither authenticates approval nor stores rejection: unchanged code/model still returns REVIEW after a human rejects the proposal. Repository controls must prevent the merge; the developer revises or withdraws the code and reruns. Persisting the rationale or adding a lasting constraint is a proposed governance action, not implemented automatic rejection handling.

6 System Architecture

Figure 1 separates approved intent from base and proposed implementation state. ArchSync Core validates the model and provides the expected graph and executable rules. Guardian caches a base Observed Graph, scopes the Git diff to affected source components, incrementally constructs the head graph, and preserves source evidence. The gate compares base and head findings before deriving the pull-request decision.

The architecture is divided across focused repositories. `archsnc-core` contains the architecture contract and deterministic conformance functions. `archsnc-guardian` contains analyzers and findings. `archsnc-benchmark` contains the controlled development benchmark. `archsnc-examples` contains readable models and generated views. `archsnc-mcp` is reserved for a later interaction layer and is excluded from the initial experiment. The named draft exposes these real repository links; the double-blind build replaces them with blinded artifact labels.

7 Implementation

7.1 Prototype and Detector Logic

`archsnc-core` implements the architecture contract, schema and semantic validation, normalized graph comparison, and deny/allow/require/require-path rules. `archsnc-guardian` implements TypeScript extraction, source evidence, and the Git-diff gate.

Guardian delegates conformance semantics to Core so repository and incremental checks use the same rule engine.

For a concrete PostgreSQL example, consider the following code inside the `order-service` component:

```
import { Pool as PgPool } from "pg";
const db = new PgPool({
  connectionString: "postgres://postgres:5432/orders"
});
await db.query("select 1");
```

The import detector records `PgPool` as a local name for a recognized `pg` constructor. The initializer pass recognizes `new PgPool`, resolves the inline `connectionString`, and associates `db` with target `postgres`, derived from the URL hostname. At `db.query`, the call detector requires that receiver association before emitting the directed edge (`order-service,data,postgres`). Merely importing `pg`, constructing an unrelated client, or calling an unrelated object's `query` method does not satisfy this detector. Construction alone does not emit the database-access edge. Evidence identifies the query call's file, line, column, detector, and excerpt.

The analysis uses per-file identifier maps and four passes over variable declarations; it is not general alias analysis or interprocedural data-flow analysis. Imported client wrappers, arbitrary reassignment, shadowed identifiers, and relationships requiring resolution across files are not comprehensively handled. Endpoint resolution uses literals, mapped variables, and naming conventions such as `DATABASE_URL` for `postgres`. For binary expressions it tries the right-hand expression first; this deterministic heuristic does not evaluate runtime environment values or branch conditions. The PostgreSQL evidence confidence of 0.99 is a fixed detector annotation, not a calibrated probability. Redis operations similarly require a recognized client, and AMQP operations require a channel associated with a recognized connection.

7.2 Controlled Evaluation Artifact

`archsnc-benchmark` supplies the synthetic Order Platform, 20 executable patches with group-authored ground truth, and the 40-signal positive/hard-negative corpus. Each patch runs on a fresh copy. Git-diff experiments additionally construct a baseline commit, perform cold and warm checks, and compare incremental head observations with a separate full scan. This full scan checks incremental equivalence; it does not independently establish extraction accuracy.

The reported measurements use pinned Core and Guardian revisions, preserved source and patch manifests, and raw result artifacts. Verification checks artifact integrity and recomputes derived results. Revision identifiers, package hashes, coverage and smoke-test counts, historical verification commands, and the code-level detector/gate audit are supplied in the supplementary implementation audit. These controls make the experiment inspectable; they are not a research contribution or evidence of accuracy on external systems. `archsnc-mcp` is not used in the experiment.

8 Controlled Verification Methodology

D1 development benchmark. An Order Platform baseline has five components and five directed relationships. Twenty isolated patches comprise nine no-impact changes, seven violations, and

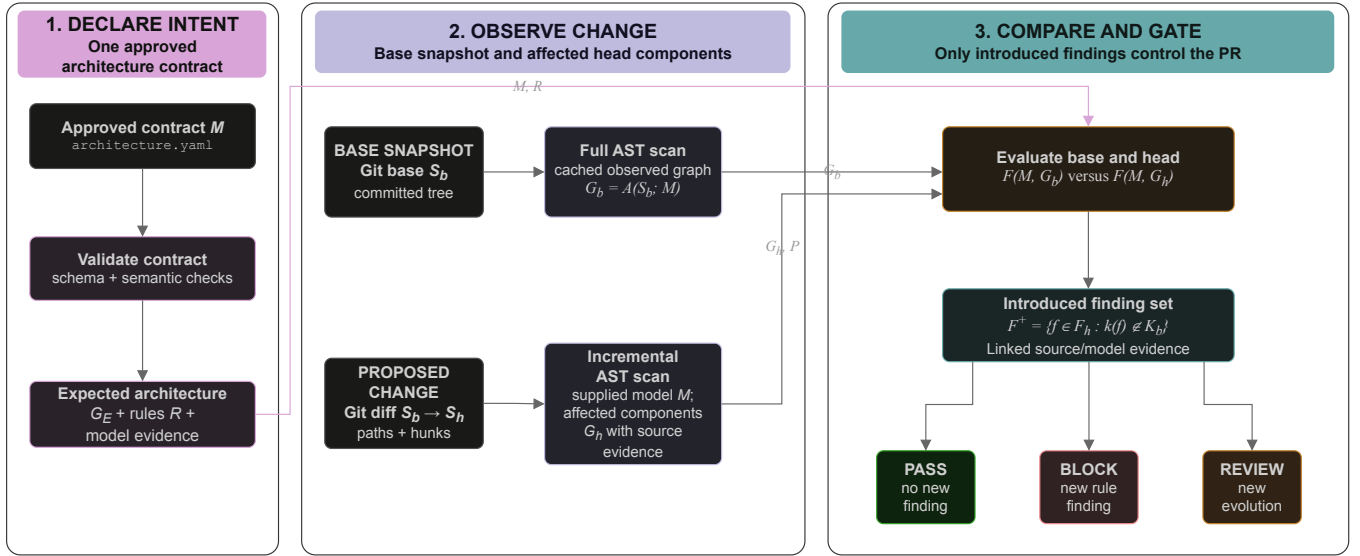


Figure 1: ArchSync Phase 3 system architecture. The three-stage pipeline separates approved intent, base/head observation, and the pull-request gate. The model remains authoritative; only findings introduced by the Git diff determine PASS, BLOCK, or REVIEW.

four evolutions. Cases include service bypass, required-edge removal, allow and require-path rules, PostgreSQL aliases, Redis and AMQP additions, a new service, and non-architectural lookalikes. Each declares changed files, graph delta, classification, rule identifiers, and source evidence. Topology intent originated before analyzer integration, but patches and exact evidence lines were refined alongside it. D1 is a co-developed regression oracle, not a blinded holdout.

D2 detector corpus. Five detector groups (HTTP, PostgreSQL, Redis, AMQP publish, and AMQP consume) each contain four positive and four hard-negative source lines: 40 annotated signals in total. Positives exercise aliases, namespaces, environment fallbacks, and supported operations. Negatives include unused imports, custom functions, method-name collisions, comments, and string literals. An annotation binds detector, file, line, expected edge, and source substring. D2 was authored from known v 0.1 weaknesses and guided v 0.2, so the paired result measures targeted regression behavior.

Execution and P3 replay. D1 validates model, manifest, hashes, and patch applicability, then executes an unchanged baseline twice and each individually patched copy twice: 42 analyzer runs. D2 executes each of two pinned analyzer versions twice. P3 creates a fresh Git baseline for every D1 patch, applies it, and runs one cold and one warm diff check. A separate full-head scan provides an equivalence oracle. The same 20 patches are reused, not counted as a third independent dataset. Normalized duplicate outputs exclude timing and cache state. The functional workflow was reproduced on clean Linux and Windows CI runners; latency comes only from the recorded Windows environment.

Measures. D1 reports matched, extra, and missed graph items, separating changed items from repeated baseline structure. Empty changed sets contribute no positive item. Classification uses all 20

cases; rule agreement uses seven violations; evidence agreement uses 11 finding-bearing cases and requires at least one matching file and line. D2 counts annotated detector signals, not unique edges, with

$$P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN},$$

$$F_1 = \frac{2PR}{P + R}, \quad Sp = \frac{TN}{TN + FP}.$$

Specificity covers only declared hard negatives. P3 checks decision, changed-file and architecture-delta equality, full-scan equivalence, replay, and expected cache miss/hit. Parsed scope is the ratio of affected-component TypeScript file instances to full-head instances. Latency medians and percentiles are descriptive; no population, significance, or comparative-performance inference is made.

Engineering gates. The existing protocol required D1 precision/recall at least 0.85, exact decisions/rules/evidence, and deterministic replay; D2 required all annotated positives and negatives correct; P3 required exact replay/full-scan agreement and cache behavior. These are development acceptance rules, not statistical standards. The paper reports counts even where its underlying engineering gate computes rates.

External-tool feasibility run. A separate frozen D1 inventory executes dependency-cruiser 18.3.0 alongside Guardian 0.3.3 and Core 0.1.1 on the baseline and 20 patches. Its macOS/Node 26 runtime and packages differ from the historical detector and Windows timing experiments. The comparator inventories TypeScript pre-compilation imports; subject package dependencies remain uninstalled, preserving unresolved references. No service rule is translated into a non-equivalent import rule. Inputs and configuration were frozen before execution. A patch-isolation error invalidated

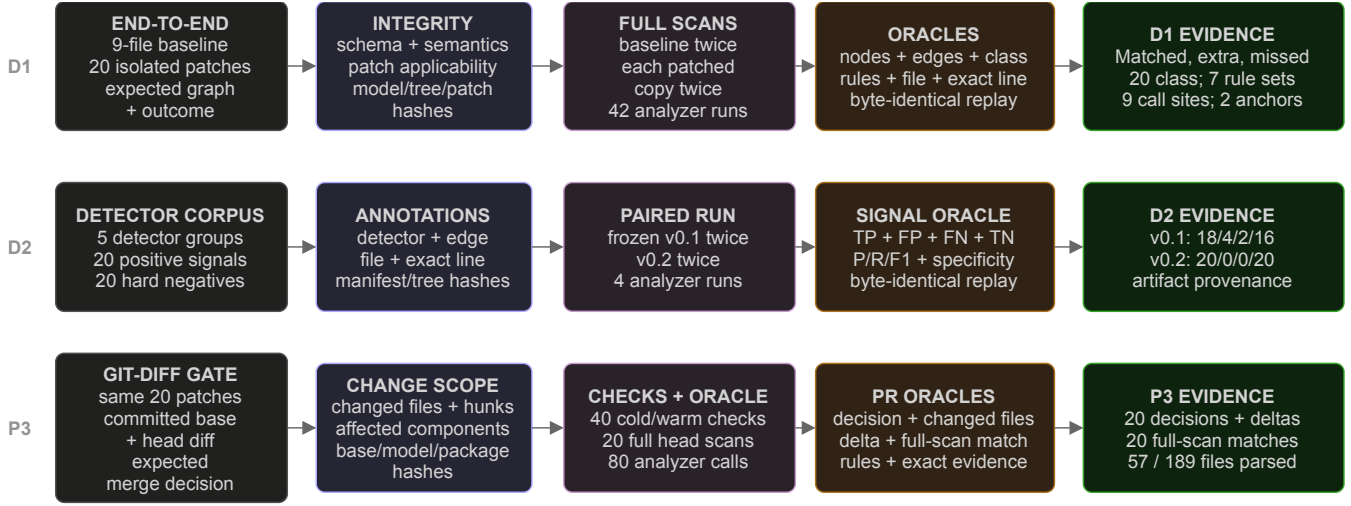


Figure 2: Controlled verification: full-graph D1 changes, D2 source signals, and incremental P3 replay. All routes use co-developed data.

the first run; a corrected fresh-Git-root procedure, exact changed-file checks, and source hashes were frozen before the valid rerun. Invalid outputs remain audit-only. This is a capability check on co-developed data, not the proposed independent D3 study.

9 Controlled Verification Results

RQ1: graph reconstruction and detector signals. The unchanged repository was reconstructed as five components and five relationships with a no-impact classification. Table 1 reports no false positive or false negative in D1. Full-graph totals aggregate 105 expected nodes and 108 expected edges across 20 patched repositories. Only five node changes and 12 edge changes contribute positive changed-graph items. These finite, co-developed cases test contract agreement; they do not estimate performance on unseen projects.

Table 4 gives the signal-level regression result. On D2, v0.1 achieved precision 0.818, recall 0.900, F1 0.857, and specificity 0.800. Its four false positives were a PostgreSQL-shaped local query, two AMQP-publish lookalikes, and an AMQP-consume lookalike; its two false negatives were Redis hSet and lPush. On the same annotated signals, v0.2 matched all 20 positives and rejected all 20 annotated negatives (precision, recall, F1, and specificity each 1.000 on this corpus). Package-provenance tracking removed the four method-name false positives, while the expanded Redis operation set recovered the two missed positives. Because D2 was constructed from known v0.1 weaknesses, this is targeted regression evidence and not an estimate over arbitrary TypeScript syntax or an external baseline comparison.

RQ2: change classification. ArchSync matched all 20 D1 labels: nine no-impact, seven violation, and four evolution cases. It also matched the complete expected rule-identifier set for all seven violations. The added allow and require-path cases exercise rule semantics beyond direct deny and require checks. Redis, AMQP, Fraud Service, and payment-cache additions were classified as evolution rather than violation, preserving a human-review path.

RQ3: evidence localization. All 11 finding-bearing D1 cases contained the expected file and at least one evidence item at the exact expected line. Violation cases pointed to either the introducing call site or, for a missing required edge or path, a deterministic component anchor. Evolution cases pointed to the operation that created the new observed relationship. The metric does not evaluate uniqueness or ranking when multiple evidence items exist.

RQ4: determinism and reproducibility. The D1 baseline and all 20 cases produced byte-identical normalized JSON across duplicate executions, giving 42 current-version executions with no replay mismatch. Both D2 analyzer versions were also deterministic across their duplicate runs. P3 produced identical normalized cold and warm results for 20/20 cases, matched all 20 declared architecture deltas, matched a separate full head scan in all 20 cases, and hit the baseline cache on every repeated check. As Table 3 shows, component-incremental analysis parsed 57 of 189 TypeScript file instances, $r_{\text{parse}} = 0.3016$. On the recorded Windows environment, warm median latency was 242.62 ms compared with 518.51 ms cold, a descriptive 53.2% reduction; warm p95 was 249.30 ms compared with 531.05 ms cold. Local verification reproduced all stored results and repository gates. GitHub Actions independently reproduced the functional benchmark, detector, and Git-diff evidence on both ubuntu-latest and windows-latest; timing values were not treated as cross-platform acceptance thresholds.

External-tool capability inventory. All 42 executions completed and all 20 patched copies matched their declared changed-file sets. Across the 20 patches (189 source-file instances), dependency-cruiser reported 54 import-dependency instances: 50 unresolved package references and four resolved local imports. Guardian reported 108 service-relationship instances. The unchanged baseline produced two imports and five service relationships. Counts include repeated structure across snapshots; they are not unique edges or interchangeable units.

D1 contains no labeled module-import relationship that both tools expose as an architecture edge. The shared labeled subset

Table 1: D1 counts on 20 co-developed patches. The five changed nodes and 12 changed edges are regression items, not independent samples of general accuracy; full-graph counts repeat baseline structure.

Measure	Matched	Extra	Missed
Full-graph nodes	105	0	0
Full-graph edges	108	0	0
Changed nodes	5	0	0
Changed edges	12	0	0

Table 2: Decision, evidence, and replay outcomes on controlled D1 cases.

Measure	Matches	Total
Classification	20	20
Violation rule set	7	7
Evidence file	11	11
Evidence exact line	11	11
Deterministic cases	20	20

Table 3: P3 Git-diff gate outcomes and measured scope on the D1 replay.

Measure	Result
Classification agreement	20/20
Merge-decision agreement	20/20
Changed-file agreement	20/20
Architecture-delta agreement	20/20
Incremental/full-scan agreement	20/20
Violation rule-set agreement	7/7
Exact evidence line	11/11
Cache hit on repeated check	20/20
Normalized cold/warm replay	20/20
Incremental files / head files	57/189
Cold median / p95 (ms)	518.51 / 531.05
Warm median / p95 (ms)	242.62 / 249.30

therefore contains zero items, making comparative precision, recall, F1, and agreement undefined. This result establishes a concrete obstacle to the proposed comparator choice, not comparative success or failure. In particular, the 108 service relationships cannot be scored as 108 dependency-cruiser misses. An external tool was executed, but the central independent-accuracy limitation remains.

Input and output digests, runtime versions, and replay commands are retained in the supplementary artifact audit. They establish which experiment was executed, not independent validity of its labels.

10 Discussion

The results establish agreement between implemented contracts and their development oracles. D2 shows why end-to-end graph counts alone are insufficient: repeated baseline structure can dominate totals, while hard-negative source locations expose method-name collisions. Its six corrected v 0.1 errors support targeted diagnosis and regression protection, not an unbiased estimate of improvement on unseen programs.

The three-way decision separates forbidden relationships from changes requiring a design decision. P3 reports historical findings

while gating only introduced findings, permitting gradual adoption without silently accepting new violations. REVIEW neither approves evolution nor rewrites the expected model. The evaluated implementation has no durable rejection disposition: rejecting an evolution requires changing or withdrawing the proposed code and rerunning the check; unchanged inputs still yield REVIEW. Accepting it requires a separately reviewed model change. Owner protection is therefore an organizational prerequisite, not an experimentally validated property of the gate.

Component scoping parsed 57/189 file instances and warm caching reduced the recorded median from 518.51 to 242.62ms. Those observations combine parsing, Git processes, cache I/O, comparison, and reporting; they do not isolate algorithmic cost or establish scale. Next experiments require independently annotated real repositories and histories, capability-aligned external baselines, and a practitioner pilot measuring false-block burden, review effort, and model-change governance. IaC, runtime, AI assistance, and other language analyzers remain unevaluated extensions.

11 Threats to Validity

Construct validity. Typed topology and explicit rules do not measure responsibility, rationale, runtime quality, or developer usefulness. Evidence agreement requires one matching location, not complete or well-ranked evidence. An absent relationship has only a deterministic component anchor. D2 signals can share an edge, and its negatives do not represent all unsupported syntax. Parsed-file counts measure analysis scope, not CPU work.

Internal validity. The authors developed the analyzer and both datasets. D1 topology intent preceded integration, but patch implementations and exact evidence lines co-evolved with the analyzer. D2 targeted known v0.1 errors and guided v0.2. Replaying D1 in P3 adds incremental checks, not independent observations. Hashes, raw predictions, mutation checks, and duplicate execution expose inconsistency but do not establish independent ground truth.

External and conclusion validity. One small synthetic system, 20 patches, and 40 curated signals cannot estimate general TypeScript accuracy. The five changed nodes, 12 changed edges, seven violation rule sets, and 11 evidence cases are finite regression items. Wrappers, dynamic endpoints, dependency injection, generated code, unconventional repositories, other languages, and runtime-only relationships remain outside evaluated coverage. Latency is descriptive from one environment; there is no workload-population inference, scalability study, or significance test. No industrial deployment or developer study was executed. The private repository plan did not enforce non-bypassable merge protection.

Table 4: D2 within-tool regression confusion matrices for the frozen v0.1 and provenance-aware v0.2 analyzers on the same co-developed 40-signal corpus.

Detector	v0.1			v0.2				
	TP	FP	FN	TN	TP	FP	FN	TN
fetch	4	0	0	4	4	0	0	4
PostgreSQL	4	1	0	3	4	0	0	4
Redis	2	0	2	4	4	0	0	4
AMQP publish	4	2	0	2	4	0	0	4
AMQP consume	4	1	0	3	4	0	0	4
Overall	18	4	2	16	20	0	0	20

Comparative validity. The executed external inventory has no common labeled units and supplies no accuracy baseline. Unresolved imports reflect uninstalled subject dependencies rather than labeled analyzer errors. Distinct observation semantics and newer pinned packages prevent its import inventory from extending the historical service-graph accuracy or timing claims. Unsupported service edges are excluded rather than counted as comparator misses. Independently authored data and capability-matched ground truth remain necessary.

Literature positioning. The narrative review carries source-selection and interpretation bias and may omit approaches. Coverage completeness and absence-of-evidence claims are outside its design, not deferred SLR deliverables. Related studies can share projects or implementations, so citation breadth does not imply independent replication.

Reliability. Pinned artifacts, normalized ordering, isolated patches, duplicate execution, and clean Linux/Windows functional checks support reproducibility. The frozen v0.1 result is provenance-validated rather than continuously regenerated; Windows timing is not a cross-platform performance claim. The external run’s invalid first attempt remains auditable and excluded, with the corrected patch-isolation procedure frozen before rerunning.

12 Conclusion and Future Work

ArchSync implements a deterministic bridge between a declared architecture contract and a source-derived TypeScript service graph. On the co-developed development datasets, its graph, classification, evidence-location, replay, and incremental/full-scan outputs agree with the declared regression oracles. The targeted detector corpus also exposes concrete errors in the earlier analyzer and verifies that provenance tracking addresses those cases.

These findings show that the current contracts, detectors, and Git-diff workflow operate coherently on their controlled scope. They do not establish general accuracy, comparative advantage, or governance effectiveness because the datasets are synthetic and co-evolved with the implementation, the executed external-tool inventory has no shared labeled units, and no independent real-world holdout exists. ArchSync keeps design changes human-governed: the analyzer never rewrites the approved model, violations block, and evolution requires review. The immediate research priorities are an independently curated multi-repository holdout, a capability-matched comparison with an existing TypeScript dependency-conformance tool, and a real pull-request

pilot. Only after those gates produce auditable evidence should the paper make claims about external accuracy, performance, or developer value.

Author Information and Contributions

Vo Duc Hieu (ORCID: 0009-0007-5389-5177) - Conceptualization; Methodology; Software; Project administration; Writing.

Tran Minh Hoang (ORCID: 0009-0000-0302-1841) - Software; AI methodology; Validation.

Ha Hoang Bach (ORCID: 0009-0000-5118-0660) - Investigation; Data curation; Formal analysis; Evaluation.

Le Van Kiet (ORCID: 0009-0007-8434-882X) - Software; Validation; Reproducibility.

References

- [1] Yalemisew Abgaz, Andrew McCarren, Peter Elger, David Solan, Neil Lapuz, Marin Bivol, Glenn Jackson, Murat Yilmaz, Jim Buckley, and Paul Clarke. 2023. Decomposition of Monolith Applications Into Microservices Architectures: A Systematic Review. *IEEE Transactions on Software Engineering* 49, 8 (2023), 4213–4242. doi:10.1109/TSE.2023.3287297
- [2] Metin Altunışık, Hasan Sözer, and Gonca Gürsun. 2024. Software Architecture Recovery from Multiple Dependency Models. In *Proceedings of the 39th ACM/SIGAPP Symposium on Applied Computing*. ACM, 1185–1192. doi:10.1145/3605098.3635917
- [3] Emilie Anthony, Astrid Berntsson, Tiziano Santilli, and Rebekka Wohlrab. 2024. We’re Drifting Apart: Architectural Drift from the Developers’ Perspective. In *2024 IEEE 21st International Conference on Software Architecture*. IEEE, Hyderabad, India, 101–111. doi:10.1109/ICSA59870.2024.00018
- [4] Alexander Bakhtin, Xiaozhou Li, Jacopo Soldani, Antonio Brogi, Tomas Cerny, and Davide Taibi. 2024. *Tools Reconstructing Microservice Architecture: A Systematic Mapping Study*. Springer Nature Switzerland, 3–18. doi:10.1007/978-3-031-66326-0_1
- [5] Tomas Cerny, Amr S. Abdelfattah, Jorge Yero, and Davide Taibi. 2024. From static code analysis to visual models of microservice architecture. *Cluster Computing* 27, 4 (April 2024), 4145–4170. doi:10.1007/s10586-024-04394-7
- [6] Clelio Orlando de Oliveira Junior, Jonathan Carvalho, Fábio Fagundes Silveira, Tiago Silva da Silva, and Eduardo Martins Guerra. 2022. CALint: A Tool for Enforcing the Clean Architecture’s Dependency Rule in Python. In *Computational Science and Its Applications – ICCSA 2022 Workshops: Malaga, Spain, July 4–7, 2022, Proceedings, Part V*. Springer International Publishing, 534–549. doi:10.1007/978-3-031-10548-7_39
- [7] Stéphane Ducasse and Damien Pollet. 2009. Software Architecture Reconstruction: A Process-Oriented Taxonomy. *IEEE Transactions on Software Engineering* 35, 4 (2009), 573–591. doi:10.1109/TSE.2009.19
- [8] Patric Genfer and Uwe Zdun. 2024. Exploring Architectural Evolution in Microservice Systems Using Repository Mining Techniques and Static Code Analysis. In *Software Architecture: 18th European Conference, ECSA 2024, Luxembourg City, Luxembourg, September 3–6, 2024, Proceedings*. Springer Nature Switzerland, 157–173. doi:10.1007/978-3-031-70797-1_10
- [9] David Kaundstorfer, Anastasia Isychev, Valentin Wüstholtz, and Maria Christakis. 2024. Interrogation Testing of Program Analyzers for Soundness and Precision Issues. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. ACM, New York, NY, USA, 319–330. doi:10.1145/3691620.3695034
- [10] Jens Knodel and Daniel Popescu. 2007. A Comparison of Static Architecture Compliance Checking Approaches. In *Sixth Working IEEE/IFIP Conference on Software Architecture*. IEEE, Los Alamitos, CA, USA, 12–21. doi:10.1109/WICS

- A.2007.1
- [11] Marco Konersmann, Angelika Kaplan, Thomas Kühn, Robert Heinrich, Anne Kozirolek, Ralf H. Reussner, Jan Jürjens, Mahmood al Doori, Nicolas Boltz, Marco Ehl, Dominik Fuchß, Katharina Großer, Sebastian Hahner, Jan Keim, Matthias Lohr, Timur Sağlam, Sophie Schulz, and Jan-Philipp Töberg. 2022. Evaluation Methods and Replicability of Software Architecture Research Objects. In *2022 IEEE 19th International Conference on Software Architecture (ICSA)*. IEEE, Honolulu, HI, USA, 157–168. doi:10.1109/ICSA53651.2022.00023
 - [12] Ruiyin Li, Peng Liang, and Paris Avgeriou. 2023. Warnings: Violation symptoms indicating architecture erosion. *Information and Software Technology* 164 (2023), 107319. doi:10.1016/j.infsof.2023.107319
 - [13] Ruiyin Li, Peng Liang, Mohamed Soliman, and Paris Avgeriou. 2022. Understanding Software Architecture Erosion: A Systematic Mapping Study. *Journal of Software: Evolution and Process* 34, 3 (2022), e2423. doi:10.1002/smr.2423
 - [14] Ruiyin Li, Mohamed Soliman, Peng Liang, and Paris Avgeriou. 2022. Symptoms of Architecture Erosion in Code Reviews: A Study of Two OpenStack Projects. In *2022 IEEE 19th International Conference on Software Architecture (ICSA)*. IEEE, 24–35. doi:10.1109/icsa53651.2022.00011
 - [15] Jingwen Liu, Wuxia Jin, Junhui Zhou, Qiong Feng, Ming Fan, Haijun Wang, and Ting Liu. 2024. 3Erefactor: Effective, Efficient and Executable Refactoring Recommendation for Software Architectural Consistency. *IEEE Transactions on Software Engineering* 50, 10 (Oct. 2024), 2633–2655. doi:10.1109/tse.2024.3449564
 - [16] Andrielly Lucena, Everton L. G. Alves, and João Brunet. 2025. Language Models as Architectural Gatekeepers: Automating Conformance Checking from Natural Language. In *Anais do XXXIX Simpósio Brasileiro de Engenharia de Software (SBES 2025)*. Sociedade Brasileira de Computação, 713–719. doi:10.5753/sbes.2025.11100
 - [17] Gail C. Murphy, David Notkin, and Kevin J. Sullivan. 1995. Software Reflexion Models: Bridging the Gap Between Source and High-Level Models. In *Proceedings of the Third ACM SIGSOFT Symposium on Foundations of Software Engineering*. ACM, New York, NY, USA, 18–28. doi:10.1145/222124.222136
 - [18] Evangelos Ntontos, Uwe Zdun, Jacopo Soldani, and Antonio Brogi. 2022. *Assessing Architecture Conformance to Coupling-Related Infrastructure-as-Code Best Practices: Metrics and Case Studies*. Springer International Publishing, 101–116. doi:10.1007/978-3-031-16697-6_7
 - [19] Abdul Qayum, Mengqi Zhang, Simon Colreavy, Muslim Chochlov, Jim Buckley, Dayi Lin, and Ashish Rajendra Sai. 2024. A Framework and Taxonomy for Characterizing the Applicability of Software Architecture Recovery Approaches: A Tertiary-Mapping Study. *Software: Practice and Experience* (2024). First published online 17 July 2024. doi:10.1002/spe.3364
 - [20] Matheus Resende, Rafael Durelli, and Ricardo Terra. 2025. ArchLintor: detecting architectural violations in TypeScript systems. In *Anais do XXXIX Simpósio Brasileiro de Engenharia de Software (SBES 2025)* (SBES 2025). Sociedade Brasileira de Computação, 900–906. doi:10.5753/sbes.2025.11075
 - [21] Simon Schneider, Alexander Bakhtin, Xiaozhou Li, Jacopo Soldani, Antonio Brogi, Tomas Cerny, Riccardo Scandariato, and Davide Taibi. 2025. Comparison of static analysis architecture recovery tools for microservice applications. *Empirical Software Engineering* 30, 5 (June 2025). doi:10.1007/s10664-025-10686-2
 - [22] Ricardo Terra and Marco Tulio Valente. 2009. A Dependency Constraint Language to Manage Object-Oriented Software Architectures. *Software: Practice and Experience* 39, 12 (2009), 1073–1094. doi:10.1002/spe.931
 - [23] Burak Uzun and Bedir Tekinerdogan. 2024. Detecting Deviations in the Code Using Architecture View-Based Drift Analysis. *Computer Standards & Interfaces* 87 (2024), 103774. doi:10.1016/j.csi.2023.103774
 - [24] Filip Zamfirov, Andrei Radulescu, Jacob Krüger, and Michel R. V. Chaudron. 2025. Lessons from Visualizing Software Architecture Structure Conformance at Thermo Fisher Scientific. In *Software Engineering and Advanced Applications: 51st Euromicro Conference, SEAA 2025, Salerno, Italy, September 10–12, 2025, Proceedings, Part III*. Springer Nature Switzerland, 372–389. doi:10.1007/978-3-032-04207-1_25
 - [25] Wenjing Zhan, Ran Mo, Yingjie Jiang, and Dongyu Wang. 2025. Just-in-Time Prediction of Software Architectural Changes Through Commit-Level Analyses. *IEEE Transactions on Software Engineering* 51, 8 (Aug. 2025), 2285–2304. doi:10.1109/tse.2025.3587849